CarpConnect Final Project Documentation

1. Project Overview

CarpConnect is a full-stack ride-sharing and carpool management platform designed for two primary user roles:

- Rider
- Driver

The platform allows riders to search for rides, create ride requests, book rides, pay through Stripe or cash-based flows, track live trips, communicate with drivers, manage subscriptions, and view emissions impact.

Drivers can publish ride offers, manage rider requests, accept or reject riders, go live during trips, track earnings, receive reviews, and monitor their own subscription and usage limits.

The system combines:

- A React + Vite frontend
- An Express + Node.js backend
- MongoDB for persistence
- Socket.IO for real-time communication
- Stripe for subscription billing and payment intent flows
- Mapping/geolocation integrations for route and nearby ride discovery

---

2. Business Goals

The product is designed to support:

- Shared mobility and carpool coordination
- Cost savings for riders and drivers
- Environmental impact tracking through emissions savings
- Subscription-based monetization
- Real-time trip operations and communication

---

3. High-Level Architecture

Frontend

Location: `client/`

Technology stack:

- React 18
- TypeScript
- Vite
- Tailwind CSS
- Shadcn/Radix UI
- Framer Motion
- Recharts
- Socket.IO client
- Stripe client libraries
- React Router

Backend

Location: `Server/`

Technology stack:

- Node.js
- Express
- Mongoose
- MongoDB
- Socket.IO
- Stripe SDK
- JWT authentication

Real-Time Layer

- Socket.IO is initialized in `Server/server.js`
- User-specific rooms and driver-specific rooms are created automatically
- Ride and chat rooms are joined dynamically

Database

- MongoDB is connected through `Server/config/db.js`
- Mongoose models define all business entities

---

4. Core User Roles

Rider

Rider capabilities:

- Search rides
- View nearby rides
- Create direct bookings
- Create ride requests
- Accept or reject counter offers
- Track ongoing rides
- View ride history
- Chat with drivers
- View wallet and emissions
- Manage subscription plan
- Rate drivers

Driver

Driver capabilities:

- Publish ride offers
- Manage incoming rider requests
- Accept, reject, or counter requests
- View active bookings
- Go live during trips
- Track earnings
- View driver ratings
- Access ride history
- Manage account and vehicle profile
- Manage subscription plan

---

5. Frontend Module Breakdown

5.1 Public Pages

These are marketing and entry pages outside the dashboards.

- `client/src/pages/About.tsx`

- About page and product/company story

- `client/src/pages/Contact.tsx`

- Contact/inquiry page

- `client/src/pages/Vision.tsx`

- Vision/mission page

- `client/src/pages/Login.tsx`

- User login screen

- `client/src/pages/Signup.tsx`

- User registration screen

5.2 Landing Page Components

Used to explain the product publicly.

- `Hero.tsx`
- `Features.tsx`
- `HowItWorks.tsx`
- `Pricing.tsx`
- `Sustainability.tsx`
- `Testimonials.tsx`
- `FAQ.tsx`
- `CTA.tsx`
- `AppDownload.tsx`
- `Footer.tsx`

Purpose:

- Present the product to visitors
- Explain plans and value proposition
- Convert visitors into registered users

5.3 Rider Dashboard

Main dashboard shell:

- `client/src/pages/Dashboard.tsx`

Key rider dashboard modules:

- `RiderOverviewCompact.tsx`

- Dashboard summary view

- Quick stats and compact activity information

- `FindRide.tsx`

- Search rides by route/date

- Nearby ride discovery

- Direct booking flow
- Stripe payment modal trigger
- Ride request creation

- `MyRides.tsx`

- Rider-side ride list
- Booking state management
- Live ride state handling

- `Community.tsx`

- Community insights and top drivers

- `RiderRatings.tsx`

- Ratings given and received

- `Emissions.tsx`

- Environmental savings dashboard

- `WalletPage.tsx`

- Spend history and rider payment summary

- `SubscriptionPage.tsx`

- Billing page for Free, Plus, and Pro plans
- Real Stripe checkout
- Test bypass button for client/testing

- `Messages.tsx`

- Booking-based chat interface

- `AccountSettings.tsx`

- Rider account settings and preferences

5.4 Driver Dashboard

Main dashboard shell:

- `client/src/pages/DriverDashboard.tsx`

Key driver dashboard modules:

- `DriverOverviewCompact.tsx`

- Driver summary and KPI view

- `MyOffers.tsx`

- Driver ride offer management

- `DriverBookings.tsx`

- Manage rider requests and accepted bookings

- `OfferRide.tsx`

- Publish new ride offers

- `LiveRide.tsx`

- Driver live trip operations

- Arrival, pickup, ride progression, and completion

- `DriverEarnings.tsx`

- Driver-side earnings summary and transaction ledger

- `DriverRatings.tsx`

- Driver reviews and compliments

- `DriverProfile.tsx`

- Public/driver profile details

- `RideHistory.tsx`

- Historical trip records for drivers

- `Messages.tsx`

- Driver-side booking chat

- `SubscriptionPage.tsx`

- Shared billing page used by both dashboards

5.5 Shared Frontend Components

- `LeafletMap.tsx`

- Map rendering

- `GoogleMap.tsx`

- Google Maps rendering helper

- `StripeCheckoutModal.tsx`

- Stripe card-payment booking flow for ride payments

- `RealTimeNotifications.tsx`

- Socket-based notification listener

- `NotificationsPanel.tsx`

- Notification display panel

- `DriverProfileModal.tsx`

- Quick driver profile modal

5.6 Frontend Utility Modules

- `client/src/lib/api.ts`

- Axios API wrapper

- `client/src/lib/plans.ts`

- Free/Plus/Pro plan definitions

- `client/src/lib/planAccess.ts`

- Plan access checks

- `client/src/lib/addressAutocomplete.ts`

- Address suggestions and geocoding support

- `client/src/lib/mapsLoader.ts`

- Loads Google Maps dynamically

- `client/src/lib/rideStatus.ts`

- Ride status formatting/normalization

---

6. Backend Module Breakdown

6.1 Server Bootstrap

- `Server/server.js`

Responsibilities:

- Loads environment variables
- Connects to MongoDB
- Creates Express app
- Configures Socket.IO
- Registers middleware
- Mounts API routes
- Starts recurring background jobs

6.2 Authentication and User Management

- `Auth.controllers.js`
- `Authroutes.js`
- `authMiddleware.js`

Responsibilities:

- Signup
- Login
- JWT token handling
- Current user fetch
- Profile updates
- Password change
- Subscription plan endpoints
- Stripe subscription checkout
- Stripe subscription sync
- Test bypass upgrade

6.3 Ride Offers

- `RideOfferController.js`
- `RideOfferRoutes.js`

Responsibilities:

- Create driver ride offers
- Update and cancel offers
- List available offers
- Get driver-owned offers

6.4 Ride Requests

- `RideRequestController.js`
- `RideRequestroutes.js`

Responsibilities:

- Rider ride requests
- Open driver request list
- Driver counter offers
- Driver rejection flow
- Rider counter-response flow

6.5 Rides / Booking Compatibility Layer

- `CompatRidesController.js`
- `RidesRoutes.js`

Responsibilities:

- Search rides
- Search by destination
- Nearby ride discovery
- Direct booking flow
- Active ride resolution

6.6 Booking Management

- `BookingController.js`
- `BookingRoutes.js`

Responsibilities:

- Create and cancel bookings
- Fetch rider and driver bookings
- Update booking statuses
- Arrival, pickup, live, completion transitions
- Spending summary
- Earnings summary
- Emissions report creation on completion

6.7 Matching

- `MatchingController.js`
- `MatchRoutes.js`

Responsibilities:

- Ride-to-request matching
- Match creation
- Match detail lookup
- Match lifecycle updates

6.8 Payments

- `PaymentController.js`
- `PaymentRoutes.js`

Responsibilities:

- Create Stripe payment intents for bookings
- Confirm payment
- Webhook processing
- Refund payment
- Driver payout initiation
- Payment status queries

6.9 Emissions

- `EmissionController.js`
- `EmissionsCompatController.js`
- `EmissionsRoutes.js`
- `EmissionRouts.js`

Responsibilities:

- Get emissions reports
- Get emissions statistics
- Get user emissions history
- Format emissions output for frontend dashboards

6.10 Chat and Notifications

- `ChatController.js`
- `ChatRoutes.js`
- `NotificationsController.js`
- `NotificationsRoutes.js`

Responsibilities:

- Booking-based chat
- Message read state
- Notifications retrieval
- Mark read / read-all flows
- Real-time event delivery

6.11 History and Reviews

- `historyController.js`

- `historyRoutes.js`
- `ReviewsController.js`
- `ReviewsRoutes.js`

Responsibilities:

- Ride history
- Booking history
- Emissions history
- Ratings history
- Create and retrieve reviews

6.12 Community / User Profile

- `UsersController.js`
- `UsersRoutes.js`

Responsibilities:

- Community data
- Public user profile access

---

7. Database Models

Main models used in the system:

- `User`

- Authentication, profile, subscription, ratings, preferences, vehicle info

- `RideOffer`

- Driver published ride listing

- `RideRequest`

- Rider ride request

- `Booking`

- Final booking between rider and driver

- `Match`

- Matching layer between requests and offers

- `Payment`

- Payment records and payout/refund state

- `EmissionReport`

- CO2 savings and environmental metrics

- `Review`

- Ratings and review comments

- `Notification`

- Notification feed and state

- Chat/message-related model(s)

- Booking-based messaging persistence

---

8. Booking Flow Explanation

Rider Direct Booking Flow

1. Rider finds a ride in `FindRide.tsx`

2. Rider chooses seats and payment method

3. Frontend calls `POST /api/rides/book-direct`

4. Backend:

- validates available seats

- validates subscription usage limits

- creates a match record

- creates a booking

- reduces seat availability on the offer

5. If payment method is Stripe:

- frontend calls `POST /api/payments/split`

- Stripe payment intent is created

- rider completes payment in Stripe modal

- frontend confirms via `POST /api/payments/confirm`

- booking becomes confirmed/processed

6. Driver receives a realtime event for the new booking

Driver Request-Based Booking Flow

1. Rider creates a ride request

2. Driver views request in `DriverBookings.tsx`

3. Driver can:

- accept

- reject

- send counter offer

4. If accepted or matched:

- booking and/or match records are updated

- payment and trip flow continue

Live Ride Flow

1. Driver marks arrival

2. Driver marks rider picked up

3. Ride becomes live

4. On completion:

- booking status updates to completed

- payment is marked processed

- emissions report is created

- driver payout logic may execute

---

9. Emissions Logic Explanation

The project tracks estimated CO2 savings from shared rides compared to solo driving.

Current baseline logic:

- baseline carbon factor: `0.171 kg CO2/km`
- solo occupancy baseline: `1.5`
- shared occupancy baseline: `1.0`

Simplified formula:

```
Savings = distanceKm x 0.171 x (1.5 - 1.0)
```

Emissions are shown in:

- Rider overview
- Emissions dashboard
- Wallet summary
- Ride completion and historical reports

Data source:

- Emissions reports are created on ride completion
- Statistics are aggregated from `EmissionReport` records

---

10. Billing and Subscription System

Plans:

- Free
- Plus
- Pro

Implementation files:

- `client/src/lib/plans.ts`
- `client/src/pages/dashboard/SubscriptionPage.tsx`
- `Server/controllers/Auth.controllers.js`
- `Server/utils/subscriptionUsage.js`

Subscription Features

- Usage caps per plan
- Stripe subscription checkout for Plus and Pro
- Stripe success/cancel redirect handling
- Subscription sync from Stripe session
- Test bypass for client/testing that upgrades current user to Pro

Usage-Limited Actions

The system enforces monthly caps for:

- ride requests
- bookings
- ride offers

Backend usage enforcement is handled by:

- `ensureSubscriptionOnUser`
- `fetchUsageCounts`
- `assertUsageAllowed`

---

11. Real-Time Features

Real-time events are managed through Socket.IO.

Primary realtime features:

- chat messaging
- booking status updates
- driver location updates
- route deviation alerts
- new booking notifications
- live ride notifications

Socket room patterns:

- `user:<userId>`
- `driver:<userId>`
- `chat:<bookingId>`
- `ride:<rideId>`

Relevant files:

- `Server/server.js`
- `client/src/components/RealTimeNotifications.tsx`
- `client/src/pages/dashboard/Messages.tsx`
- `client/src/pages/dashboard/LiveRide.tsx`
- `client/src/pages/dashboard/MyRides.tsx`
- `client/src/pages/dashboard/DriverBookings.tsx`

---

12. API Summary

Authentication

Base: `/api/auth`

Main endpoints:

- `POST /signup`
- `POST /login`
- `GET /me`
- `PATCH /profile`
- `PATCH /change-password`
- `GET /subscription/plans`
- `POST /subscription/checkout`
- `POST /subscription/sync`
- `POST /subscription/cancel`
- `POST /subscription/dev-upgrade`

Bookings

Base: `/api/bookings`

Main endpoints:

- `POST /`
- `GET /`
- `GET /:id`
- `DELETE /:id`
- `PATCH /:id/status`
- `PATCH /:id/arrived`
- `PATCH /:id/picked-up`
- `PATCH /:id/completed`
- `GET /summary/spending`
- `GET /summary/earnings`

Payments

Base: `/api/payments`

Main endpoints:

- `POST /split`
- `POST /confirm`
- `GET /:paymentId`
- `POST /refund/:paymentId`
- `POST /webhooks/stripe`
- `POST /account`

- `POST /payout/:paymentId`

Ride Offers

Base: `/api/rides/offers`

Main endpoints:

- `GET /`
- `GET /me`
- `POST /`
- `GET /:id`
- `PATCH /:id/status`
- `PUT /:id`
- `POST /:id/cancel`

Ride Requests

Base: `/api/rides/requests`

Main endpoints:

- `POST /`
- `PUT /:id`
- `GET /me`
- `GET /driver/open`
- `POST /:id/reject`
- `POST /:id/counter`
- `POST /:id/counter/respond`
- `POST /:id/cancel`

Rides Compatibility / Search

Base: `/api/rides`

Main endpoints:

- `GET /active`
- `GET /offers`
- `GET /search-dest`
- `POST /book-direct`

Chat

Base: `/api/chat`

Main endpoints:

- `POST /`
- `GET /:bookingId`
- `DELETE /:messageId`
- `POST /:bookingId/read`

Emissions

Base: `/api/emissions`

Main endpoints:

- `GET /me`
- `GET /history`
- `GET /:rideId`

Notifications

Base: `/api/notifications`

Main endpoints:

- `GET /`
- `PATCH /read-all`
- `PATCH /:id/read`

Reviews

Base: `/api/reviews`

Main endpoints:

- `POST /`
- `GET /user/:userId`
- `GET /history`
- `GET /:id`
- `PUT /:id`
- `DELETE /:id`

History

Base: `/api/history`

Main endpoints:

- `GET /rides`
- `GET /bookings`
- `GET /emissions`

- `GET /ratings`
- `GET /ride/:id`
- `PATCH /bookings/:id/hide`
- `PATCH /bookings/clear`
- `PATCH /rides/:id/hide`
- `PATCH /rides/clear`

Community / Users

Base: `/api/users`

Main endpoints:

- `GET /community`
- `GET /:id/profile`

---

13. Environment Variables and API Keys Used

Backend Environment Variables

Used in `Server/.env`

- `PORT`

- backend server port

- `NODE_ENV`

- runtime mode

- `MONGODB_URI`

- MongoDB database connection

- `JWT_SECRET`

- JWT signing secret

- `JWT_EXPIRES_IN`

- token expiry

- `REFRESH_TOKEN_SECRET`

- refresh token secret

- `CLIENT_URL`

- client origin

  - `SERVER_URL`

- backend origin

  - `FRONTEND_URL`

- frontend redirect base for Stripe checkout

  - `STRIPE_SECRET_KEY`

- Stripe secret API key

  - `STRIPE_PUBLISHABLE_KEY`

- server-stored Stripe publishable value

  - `STRIPE_WEBHOOK_SECRET`

- Stripe webhook signature validation

  - `STRIPE_PRICE_PLUS`

- Stripe recurring price ID for Plus plan

  - `STRIPE_PRICE_PRO`

- Stripe recurring price ID for Pro plan

  - `ALLOW_DEV_STRIPE_BYPASS`

- enables backend test plan bypass

  - `AUTH0_SECRET`
  - `AUTH0_BASE_URL`
  - `AUTH0_CLIENT_ID`
  - `AUTH0_ISSUER_BASE_URL`

- Auth0-related configuration

  - `MAPBOX_API_KEY`

- reserved for route/map provider support

Frontend Environment Variables

Used in `client/.env`

- `VITE_API_URL`

- frontend API base URL

- `VITE_GOOGLE_MAPS_API_KEY`

- Google Maps JS usage

- `VITE_STRIPE_PUBLISHABLE_KEY`

- frontend Stripe publishable key

- `VITE_DEV_PLAN_BYPASS`

- frontend toggle for showing test bypass button

External Services Used

- MongoDB Atlas
- Stripe
- Google Maps
- OpenStreetMap / Nominatim
- Socket.IO
- Auth0-related placeholders/configuration

---

14. Security Notes

- JWT-based authentication is used for protected APIs
- Socket authentication uses JWT in handshake auth
- Stripe webhook route is mounted before JSON parsing for signature validation
- Protected routes use middleware to validate access
- Role-based checks are applied for rider-only and driver-only routes

Recommended production hardening:

- move all secrets to a secure deployment secret manager
- rotate exposed test keys before production launch
- restrict CORS origins to production domains only
- implement rate limiting
- add audit logging for payment/refund/admin-sensitive actions

---

15. Deployment Notes

Typical deployment split:

- Frontend:

- Vercel / Netlify / static hosting

- Backend:

- Node server on VPS / Render / Railway / similar

- Database:

- MongoDB Atlas

Deployment checklist:

- configure frontend production URL
- configure backend production URL
- set all JWT secrets
- set MongoDB URI
- set Stripe keys and webhook secret
- configure allowed CORS origins
- test Socket.IO connectivity
- test Stripe checkout and webhook callbacks

---

16. Current Product Strengths

- Full dual-role workflow for riders and drivers
- Rich dashboard-based UX
- Realtime chat and ride updates
- Subscription monetization
- Environmental impact tracking
- Historical trip and financial summaries
- Nearby ride search and map support

---

17. Known Technical Considerations

- There are some legacy/compatibility controllers and routes still present

- for example emissions and rides compatibility layers

- The codebase contains both old and new flow variants in some places
- Additional backend integration tests would improve long-term stability
- Some areas still rely on derived calculations instead of a single unified source of truth

These do not prevent the product from functioning, but they are useful future cleanup opportunities.

---

18. Recommended Future Enhancements

Product Enhancements

- in-app payout dashboard for drivers
- admin panel
- coupon/promo system
- recurring billing management portal
- saved favorite routes
- driver/rider verification workflows
- SOS / emergency contact integration
- better route optimization and ETA prediction
- waitlist and surge-based ride suggestions
- trip rescheduling flows

Technical Enhancements

- add automated integration testing for booking/payment flows
- unify legacy and compatibility controllers
- centralize emissions calculation logic in one shared module
- centralize payment status transition rules
- add proper job queues for background tasks
- improve websocket reconnect/session persistence
- add monitoring and structured logging
- split frontend bundle for performance

Analytics Enhancements

- conversion analytics from landing page to signup
- billing funnel analytics
- driver supply vs rider demand heatmaps
- emissions dashboards by city/region

---

19. Suggested Client Handover Notes

When presenting this project to the client, it is recommended to explain it as:

- a complete ride-sharing and carpool platform
- built with separate rider and driver operational dashboards
- including live trip handling, chat, billing, and sustainability tracking
- with room for future scale through modular backend controllers and route-based APIs

Key client-facing selling points:

- supports both rider and driver journeys end to end
- includes monetization through subscription billing
- includes sustainability reporting for brand value and impact visibility
- supports realtime communication and ride monitoring

---

20. Final Summary

CarpConnect is a full-featured ride-sharing application with:

- modern frontend architecture
- modular backend APIs
- real-time trip communication
- booking and payment flows
- plan-based subscription access
- emissions and sustainability reporting
- role-based dashboards for riders and drivers

This document can be used as the final client handover reference for technical overview, product structure, integrations, and future roadmap.